

P3440R2

Add `mask_from_count` function to `std::simd`

Published Proposal, 2026-02-20

This version:

<http://wg21.link/P3440R2>

Author:

[Daniel Towner](#) (Intel)

Audience:

LEWG

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Abstract

Proposal to add `std::simd::mask_from_count` function to create a mask containing an exact number of set bits. Such a function is notably useful for handling loop remainders.

Table of Contents

1	Revision History
1.1	R2
1.2	R1
2	Motivation
3	Exploration of design decisions
3.1	Free Function
3.2	Precondition and Behaviour

- 3.2.1 Performance considerations
- 3.3 Naming
- 3.4 Simd-Generic Design
- 3.5 Usage Patterns
- 3.6 Design Alternative: Range-Based Version

4 Implementation Experience

5 Wording

- 5.1 Add to [`simd.general`]
- 5.2 Add new section [`simd.mask.from_count`]

§ 1. Revision History

§ 1.1. R2

- Changed function name from `n_elements` to `mask_from_count` to give it a clearer intent.
- Explored and Clarified precondition behavior.
- Changed to simd-generic free function accepting vec/scalar types (not mask types) for natural usage in generic code.
- Added performance considerations discussion.
- Enhanced motivation section with stronger arguments against manual mask generation approaches.
- Added design alternative for range-based version.

§ 1.2. R1

- Freshened up the wording to match the current state of the draft proposal.

§ 2. Motivation

When iterating over large dynamic data sets using `std::simd` there will inevitably be situations where the very last block of data doesn't fill the entire `std::simd` object. This remainder needs to be processed using a partially filled `std::simd` object. For example:

```
void fn(std::span<float> data)
{
    using V = simd::vec<float>;
    auto count = data.size();

    // Process complete SIMD blocks.
    auto wholeBlocks = count / V::size();
    for (int i = 0; i < wholeBlocks; ++i)
    {
        auto block = simd::unchecked_load<V>(data.subspan(i * V::size()));
        process(block); // Process an entire simd-worth of data.
    }

    // Process the remainder.
    auto remainder = count % V::size();
    if (remainder > 0)
    {
        auto remainderBlock = simd::partial_load<V>(data.last(remainder));
        auto remainderMask = simd::mask_from_count<V>(remainder);
        process(remainderBlock, remainderMask); // Do the work on part of the
    }
}
```

In this example the remainder has been handled by creating a mask in which only the bits `[0..remainder)` are active. Note that the partial load of the remainder has been handled using the `partial_load` function which is memory-safe and likely to be efficiently implemented. However, the processing itself is taking a `basic_vec` and only operating on the subset of its elements which correspond to the remainder, and for this processing a suitable remainder mask must still be generated.

Beyond loop remainders, `mask_from_count` is useful whenever `std::simd` values are generated or computed rather than loaded from memory, including cryptographic block operations, algorithmic computations (generated sequences, transforms), and operations on previously-loaded data.

Without `mask_from_count`, there are several ways to create that mask, three variants of which are illustrated here:

```
int numRemainderBits = ... ;

// This is quite compact, but will have some runtime conversion to deal
// with the `float` comparison.
auto remainder1 = simd::iota<vec<float>>> < numRemainderBits;

// Like the previous, but explicitly avoid the runtime conversion to float.
auto tmp = simd::iota<vec<uint32_t>>> < numRemainderBits; // Create an n-el
auto remainder2 = mask<float>(tmp); // Convert to the correct type of mask

// Use the facilities of new mask to build directly from an integer bit set
// generates efficient code on compact mask machines (e.g., Intel AVX-512,
// It doesn't handle masks containing more than 64 elements without a change
auto m = (uint64_t(1) << numRemainderBits) - 1;
auto remainder3 = mask<float>(m);
```

One serious issue with this selection of methods is that there is no single obvious style to use to generate the best code across a range of targets. For example, the last method works well on compact-mask targets (e.g., Intel AVX-512), but poorly on wide-mask targets (e.g., Intel SSE). Adding conditional code around the mask to reflect on the target and generate the mask differently just leads to a reduction in portability and an increase in verbosity.

Manual mask generation introduces subtle correctness issues for corner cases. The integer bit-manipulation approach only works if the integer type is large enough - using a `uint16_t` value to generate a 64-bit mask will silently fail on some targets. The `iota`-based approaches also have a serious problem: `iota<vec<int8_t>>> < n` wraps at 128, silently producing wrong results if the mask has more than 128 elements. This compiles without issue, but fails at runtime, which is easy to miss in code review, and creates future portability problems as implementations support larger vectors. Users must carefully choose the `iota` element type based on the maximum possible mask size they might encounter, a requirement that is both easy to get wrong and difficult to validate.

To avoid the portability and correctness issues with manual mask generation, we propose that a free function is provided which creates a mask with exactly `N` bits active at positions `[0..N)`. By making

this function part of `std::simd` itself, the implementation can choose the most efficient implementation for the target and correctly handle all possible corner cases:

```
template<simd-type V>
constexpr /* V's mask type */ mask_from_count(simd-size-type count) noexcept
```

We also propose that a variant of this function should exist to handle the `simd-generic` case which is a design philosophy of `std::simd`.

§ 3. Exploration of design decisions

§ 3.1. Free Function

Following `std::simd`'s design principle, all operations that can be free functions are free functions. This makes the free function choice natural and consistent with the overall design, and also allows `simd-generic` style usage, which would be impossible were the function a member of `basic_vec`.

§ 3.2. Precondition and Behaviour

The function has only one precondition: `count >= 0` because negative values are meaningless and indicate a programming error.

Other preconditions have been discussed, but have been rejected. For all non-negative counts, the function can behave gracefully without resorting to preconditions:

- `count == 0` returns an empty mask
- `0 < count < size()` returns a partial mask with the first `count` elements true
- `count >= size()` saturates to a full mask

The avoidance of unnecessary preconditions maintains semantic consistency with `partial_load` and `partial_store` which similarly accept empty ranges, partial ranges, and oversized ranges without precondition violations. Furthermore other C++ standard functions dealing with potential partial operations do not impose preconditions either (e.g., `std::partial_sort` accepts `middle == first` or `middle == last`, or `std::copy_n` allows `N` to be 0). The "partial" convention established by these indicates that their ranges "*might* be partial", not "*must* be partial", and allows these functions to handle empty, partial, and full cases uniformly without forcing users to write edge-case code.

§ 3.2.1. Performance considerations

One could argue that stricter preconditions (such as requiring `0 < count < size()`) could enable micro-optimizations by eliminating edge case handling. In theory, this could allow implementations to skip checks for zero or oversized counts.

In practice, Intel's implementation experience shows no notable performance difference between implementations that handle all non-negative counts versus those with stricter preconditions. Modern compilers effectively optimize the edge case handling, and the actual mask generation dominates any conditional overhead.

The usability benefit of graceful edge case handling outweighs any theoretical performance advantage from stricter preconditions. Users can write simpler code without defensive checks, which is consistent with the design philosophy of `partial_load` and the broader C++ libraries.

§ 3.3. Naming

The chosen name `mask_from_count` makes the intent immediately clear while being reasonably concise.

Alternative names considered and rejected:

- `first_n_elements`
- `set_first_n_elements`
- `set_mask_n`
- `set_first_n`
- `mask_of_n`
- `mask_n`
- `mask_with_n_set`
- `first_n_bits_of_mask`
- `first_n_of_mask`

§ 3.4. Simd-Generic Design

Following a pattern established in `std::simd` (e.g., `select`), this function is simd-generic, accepting both `basic_vec` types and scalar types as its template parameter. This design choice enables nat-

ural usage in generic code.

The template parameter is the **vec or scalar type** (not the mask type), which is the natural parameter to work with in generic code. The function returns the appropriate mask type:

- For `basic_vec<T, Abi>` → returns `typename basic_vec<T, Abi>::mask_type`
- For scalar type `T` → returns `bool`

For scalar types the function works in a natural way: `mask_from_count<float>(count)` returns `bool(count > 0)`. This makes the scalar case consistent with the intent of vector case of deciding which elements need to be processed.

Note that if stricter preconditions were added for empty or saturated cases (e.g., requiring `0 < count < size()`), the simd-generic design would become largely meaningless. For example, with such a precondition, `mask_from_count<float>(0)` would be ill-formed, as would `mask_from_count<float>(1)` (since for scalars, `count >= size()` would violate the precondition). This would make it impossible to write generic code that works uniformly across scalar and vector types without explicit type-based branching to handle edge cases, which is precisely the verbosity and error-prone code that the simd-generic design aims to eliminate. The graceful handling of empty and saturated cases is essential to making simd-generic usage practical.

§ 3.5. Usage Patterns

The absence of preconditions on empty or full cases enables simple, uniform code that mirrors the usage of `partial_load`:

Pattern A - Unified loop (maximally simple):

```
for (int i = 0; i < count; i += simd::vec<float>::size()) {  
    auto block = partial_load<simd::vec<float>>(data.subspan(i));  
    auto mask = mask_from_count<simd::vec<float>>(count - i);  
    process(block, mask); // Same code path for all iterations  
}
```

Notice how both `partial_load` and `mask_from_count` allow the count to vary between 0 and `count` without requiring defensive checks. The loop condition `i < count` naturally prevents execution when no elements are present.

Pattern B - Explicit remainder (potentially optimized):

```
auto fullBlocks = count / simd::vec<float>::size();
auto remainder = count % simd::vec<float>::size();

for (int i = 0; i < fullBlocks; ++i) {
    auto block = unchecked_load<simd::vec<float>>(data.subspan(i));
    process(block, mask<float>(true)); // Explicit full mask
}

if (remainder > 0) {
    auto block = partial_load<simd::vec<float>>(data.last(remainder));
    auto mask = mask_from_count<simd::vec<float>>(remainder);
    process(block, mask); // Explicit partial mask
}
```

Pattern B may enable better optimization by clearly separating full blocks from the remainder, and it makes the programmer's intent more explicit. However, Pattern A's uniformity is valuable when code simplicity is prioritised.

Both patterns are equally supported. The design philosophy of matching the precondition behaviour of `partial_load` and `partial_sort` avoids forcing users into explicit edge-case handling to deal with preconditions.

§ 3.6. Design Alternative: Range-Based Version

An alternative design could provide a range-based version that mirrors `partial_load`'s call-site symmetry:

```
template<simd-type V, ranges::sized_range R>
constexpr /* V's mask type */ mask_from_range(R&& r) noexcept {
    return mask_from_count<V>(ranges::size(r));
}

// Usage:
auto block = simd::partial_load<V>(data.subspan(i, blockSize));
auto mask = simd::mask_from_range<V>(data.subspan(i, blockSize));
```


This is not proposed because it's a trivial wrapper users can easily write themselves, and unlike `partial_load`, it cannot deduce the template argument. It could be added in a future revision if usage experience demonstrates value.

§ 4. Implementation Experience

Intel's implementation of `std::simd` has had this function (albeit as a named constructor) since very early on, and it is used throughout our example code base. It makes generating efficient mask remainders across all Intel targets efficient and easy, and it makes the code's intent very obvious.

§ 5. Wording

§ 5.1. Add to `[simd. general]`

Add to the list of simd operations:

```
template<typename V>
constexpr /*see below*/ mask_from_count(simd-size-type count) noexcept;
```

§ 5.2. Add new section [**simd.mask.from_count**]

◆ Creating masks from element count [**simd.mask.from_count**]

```
template<typename V>  
constexpr /*see below*/ mask_from_count(simd-size-type count) noexcept;
```

Mandates: *V* is either a cv-unqualified vectorizable type or a specialization of `basic_vec`.

Preconditions: `count >= 0`.

Returns:

- If *V* is a specialization of `basic_vec<T, Abi>`, returns a mask object of type `basic_mask<T, Abi>` where the value of the i^{th} element is `i < count` for all *i* in the range `[0, V::size())`.
- Otherwise, returns a value of type `bool` equal to `count > 0`.